

## Research Discovery + Retrieval Platform

## Finalized Project Decisions & Immediate Action Plan

**Project positioning:** Build a research discovery and retrieval platform with an AI interface—not merely a RAG chatbot. The core value is reliable paper ingestion, deduplication/version resolution, retrieval quality, measurable evaluation, and production-style observability.

## 1. Finalized Scope

| Keep / Build        | Decision                                                      |
|---------------------|---------------------------------------------------------------|
| Paper sources       | Start with arXiv + OpenAlex                                   |
| Paper ingestion     | Normalize metadata, validate records, handle failures         |
| Deduplication       | Cross-source duplicate detection                              |
| Version resolution  | Handle arXiv revisions as versions of one logical paper       |
| Document processing | PDF → structured/section-aware text                           |
| Retrieval           | Hybrid keyword + vector retrieval, with reranking if feasible |
| RAG chatbot         | Grounded answers with paper/section evidence                  |
| Research news       | Short news-style summaries for new papers                     |
| Evaluation          | Labeled retrieval benchmark; Recall@K and nDCG@K              |
| Experimentation     | Compare at least two chunking/retrieval strategies            |
| Observability       | P50/P95 latency, failures, ingestion throughput               |
| Cost tracking       | Track embedding/LLM cost or token usage                       |

## 2. Final Architecture

```

arXiv / OpenAlex
↓
Ingestion Workers
↓
Normalize + Deduplicate + Version Resolution
↓
MySQL (metadata) PDF/Object Storage
↓
Document Parser
↓
Section-aware Chunking
↓
████████████████████████████████████████████████████████████████████████████████
↓ ↓
Vector Index BM25/Keyword
████████████████████████████████████████████████████████████████████████████████
↓
Hybrid Retrieval
↓
Reranking
↓
RAG / LLM
██ ██
Research News Chatbot

```

+ Evidence  
Parallel systems:  
Evaluation → Recall@K / nDCG@K / failure analysis  
Observability → P50/P95 latency / errors / throughput / cost

### 3. Tech Stack

| Layer               | Choice                                                                         |
|---------------------|--------------------------------------------------------------------------------|
| Frontend            | React / Next.js                                                                |
| Backend             | FastAPI                                                                        |
| Relational database | MySQL (chosen because of familiarity)                                          |
| Vector database     | Qdrant, locally for development                                                |
| Keyword retrieval   | MySQL full-text search initially, or a dedicated search engine later if needed |
| LLM / AI            | Gemini API through Google AI Studio, subject to current model/quota limits     |
| Background jobs     | Redis + Celery/RQ or a simpler worker approach initially                       |
| Containers          | Docker                                                                         |
| Evaluation          | Python + custom benchmark/evaluation scripts                                   |
| Observability       | OpenTelemetry/metrics/logging as complexity warrants                           |

### 4. RAG Evaluation Plan

Create a labeled set of questions for a representative collection of papers. For each question, identify the relevant section/chunk(s). Run multiple retrieval configurations and report Recall@K and nDCG@K. Do not invent benchmark values; record measured results.

Minimum comparison: fixed-size chunking vs section-aware chunking. Stronger comparison: add hybrid retrieval and/or reranking. Track retrieval failures and categorize why relevant evidence was missed.

### 5. Deduplication / Version Resolution

Use a staged approach: exact identifiers such as DOI/arXiv ID → normalized title/author metadata → similarity matching when identifiers are missing. Explicitly test edge cases and avoid false merges. Represent arXiv revisions as versions of one logical paper.

### 6. Performance & Cost Measurements

**Ingestion path:** fetch, parsing, chunking, embedding, indexing, failures, throughput.

**Query path:** query embedding, retrieval, reranking, LLM generation, total latency.

**Report:** P50/P95 latency and actual token/API usage or cost. Establish a realistic budget after measuring the baseline.

### 7. MCP Decision

MCP is optional and should not be the core architecture. If added, expose research tools such as `search_papers()`, `get_paper()`, `find_similar_papers()`, or `search_author()`. RAG remains responsible for retrieval and grounded answering.

### 8. Immediate Build Order

1. Define the MySQL schema for papers, authors, sources, versions, ingestion status, and processing status.
2. Implement arXiv ingestion and normalize its metadata.
3. Implement OpenAlex ingestion and normalize it into the same schema.

4. Build duplicate detection and arXiv version resolution.
5. Download/store a manageable paper corpus and build PDF → structured text extraction.
6. Implement fixed-size chunking as the baseline.
7. Add section-aware chunking.
8. Add embeddings + Qdrant and a basic vector retriever.
9. Add keyword retrieval and combine it with vector retrieval.
10. Build a small labeled evaluation set and measure Recall@K/nDCG@K.
11. Build the RAG chatbot with evidence/citations to the paper.
12. Add news-style paper summaries.
13. Instrument ingestion/query latency, errors, throughput, and model/token usage.
14. Only after the core system is reliable, consider reranking, MCP tools, and additional sources.